

math_13

March 9, 2025

```
[5]: # Подключим нужные для базовых операций библиотеки
import seaborn as sb
from matplotlib import pyplot as plt
import numpy as np
import pandas as pd

# Подключим пакеты для использования OLS метода и тестов
import statsmodels.api as sm
from sklearn.linear_model import LinearRegression
from scipy import stats
from statsmodels.stats.outliers_influence import □
    variance_inflation_factor

# Подгрузим полезные функции
from utils import *

# Сделаем автоподгрузку всех изменений при перепрогонке ячейки
%load_ext autoreload
%autoreload 2
```

The autoreload extension is already loaded. To reload it, use:
%reload_ext autoreload

```
[6]: # Определим параметры выборки для задачи пропущенной переменной
# Создадим удобный словарь, чтобы передавать его в функцию
dist_params = dict(

    # Зададим параметры распределения факторов
    x1_mean = 25.0,
    x1_std = 10.3,
    x2_mean = 20.0,
    x2_std = 5.0,
    x3_mean = 30.0,
    x3_std = 8.0,
    corr_12 = 0.0,
    corr_23 = 0.0,
    corr_13 = 0.8,
```

```

# Зададим параметры распределения ошибки
e_mean = 0.0,
e_std = 30.0,

# Укажем размер выборки
N = 1000,

# Зададим действительные параметры модели
beta0 = 100.0,
beta1 = 3.7,
beta2 = 6.3,
beta3 = -7.7
)

# Установим стартовую точку для алгоритма генерации случайных чисел
RANDOM_SEED = 42

```

```

[ ]: import numpy as np
import statsmodels.api as sm

n_sim = 10000
cover_b1 = 0
cover_b2 = 0
cover_b3 = 0

beta1_true = dist_params['beta1']
beta2_true = dist_params['beta2']
beta3_true = dist_params['beta3']

for i in range(n_sim):

    dt = gen_data(y_type='multivariate', params=dist_params,
    ↪seed=None)

    X = dt[['x1', 'x2', 'x3']]
    y = dt['y']

    X = sm.add_constant(X)

    model = sm.OLS(y, X).fit()

    ci = model.conf_int(alpha=0.05)

    ci_x1 = ci.loc['x1']

```

```

ci_x2 = ci.loc['x2']
ci_x3 = ci.loc['x3']

if beta1_true >= ci_x1[0] and beta1_true <= ci_x1[1]:
    cover_b1 += 1

if beta2_true >= ci_x2[0] and beta2_true <= ci_x2[1]:
    cover_b2 += 1

if beta3_true >= ci_x3[0] and beta3_true <= ci_x3[1]:
    cover_b3 += 1

coverage_b1 = cover_b1 / n_sim
coverage_b2 = cover_b2 / n_sim
coverage_b3 = cover_b3 / n_sim

print(f"Доля попаданий beta1 в 95%-ДИ: {coverage_b1:.3f}")
print(f"Доля попаданий beta2 в 95%-ДИ: {coverage_b2:.3f}")
print(f"Доля попаданий beta3 в 95%-ДИ: {coverage_b3:.3f}")

```

```

Доля попаданий beta1 в 95%-ДИ: 0.953
Доля попаданий beta2 в 95%-ДИ: 0.948
Доля попаданий beta3 в 95%-ДИ: 0.952

```

0.1 Выводы

Доверительные интервалы, построенные для коэффициентов регрессии, демонстрируют покрытие, близкое к заявленным 95%. Это означает, что при многократном повторении эксперимента истинные значения параметров действительно попадают в 95%-доверительные интервалы примерно в 95% случаев, как и предсказывает теория.